DAIRM: Divyendu's Data-State-Aware AI Reference Model for Generative AI Inference Systems

Divyendu Bhatt
Independent Researcher
Divyendu.bhatt@gmail.com

10 November 2026

Abstract

A generative AI request crosses a client application, an orchestration gateway, context services such as retrieval and tools, an inference-serving engine, a model-execution runtime, device drivers and accelerator hardware. Each tier has its own tools, telemetry and vocabulary, and practitioners lack a shared way to state *where* on this path a latency regression, a quality failure or a security threat originates. Several OSI-inspired layer models for AI already exist; this paper does not claim the layered-stack idea or the OSI analogy as new. It presents DAIRM, an informative reference model for the *request path of generative AI inference serving*, with a six-layer request path (application, orchestration and gateway, serving engine, model-execution runtime, device and driver, physical compute and interconnect) and three planes beside it (context services, build/deploy and management). Its contributions are limited to three items: (i) a named, observable data state at every boundary of the request path, from the request payload to the device instruction and memory-transaction stream and back as a token stream; (ii) a boundary-attributed latency diagnostic method that separates context-service delay, serving-engine queueing or KV-cache pressure, and memory-bandwidth-bound decode; and (iii) a layer-indexed telemetry attribute set that makes the method executable on OpenTelemetry traces. We compare DAIRM with related standards, frameworks and layer models, including an earlier, unrelated model that shares this work's former name, illustrate the method on a constructed example, and report a small measured study on a CPU-only serving stack (llama.cpp, a 0.5B-parameter model, 3 repetitions): an automated encoding of the triage attributed 12 of 12 injected single faults at the context service, gateway, serving engine and execution hardware to the layer where they were injected, with no false attributions in 3 no-fault controls, and in 6 of the 12 runs the boundary-state check corrected a "first span that grew" attribution to the serving engine. The study does not test the GPU-specific discriminators, and DAIRM has not been validated on production systems or adopted by any standards body. We close with an evaluation plan.

Keywords: reference model, generative AI, LLM inference serving, observability, OpenTelemetry, KV cache, layered architecture

1 Introduction

Production generative AI is no longer a model behind an API. Serving a chat completion with retrieval-augmented generation (RAG) [1] involves an application client, an API or AI gateway that applies policy and templates, a retrieval service with an embedding model and vector index, a serving engine that batches requests and manages a key–value (KV) cache [2, 3], a runtime that executes the model graph as kernels [4], a device driver, and accelerators whose memory bandwidth

often bounds decode speed [5, 6]. These tiers are usually owned by different teams, observed with different tools, and described with different words.

The result is a recurring practical problem: *there is no shared vocabulary for locating a failure or threat along the inference request path.* A report that "the model is slow" may mean retrieval latency, queueing in the serving engine, KV-cache preemption, or memory-bandwidth-bound decode; "the model hallucinated" may mean a retrieval miss, a stale index, a prompt-template defect, or a quantisation regression. Sculley et al. observed that ML systems erode abstraction boundaries and accumulate system-level debt [7]; generative inference stacks inherit that problem with more tiers.

Computer networking addressed an analogous problem with the OSI Basic Reference Model [8–10]: ordered layers, service boundaries between adjacent layers, and named data units at each boundary. This paper borrows that *method*—not the OSI layers—for generative AI inference. Many authors have drawn an OSI analogy for AI before (Section 3), including a model published under the name "AOSI" [11]; the present work was first released under that same name, developed independently, and renamed DAIRM to avoid confusion (Section 3.1). We therefore make no claim about the layered-stack idea, the number of layers or the OSI analogy. Our contributions are limited to:

1. **A named, observable data state at every boundary of the inference request path** (Section 4.4). Each state can be captured or measured: above the serving engine it is scoped to one request, and from the serving engine down, where batching mixes requests, it is scoped to one scheduler step or device and joined to its requests through a step identifier. This makes each boundary a checkpoint for attributing cause. Activations and the KV cache are treated as resident device-memory state, and weights, tokenizers and compiled graphs as build/deploy artifacts, rather than as data handed between layers.
2. **A boundary-attributed latency diagnostic method** (Section 5) combining a latency decomposition, per-layer trace attribution and single-variable isolation tests, with discriminators for three commonly confused causes: context-service delay, serving-engine queueing or KV-cache pressure, and memory-bandwidth-bound decode.
3. **A layer-indexed telemetry attribute set** (`dairm.*`, Section 5.5) that makes the method executable on OpenTelemetry traces alongside the OpenTelemetry generative AI semantic conventions [12].

We also give a structured comparison with related work (Table 1), a layer-indexed mapping of the OWASP Top 10 for LLM Applications (offered as content, not as a novelty claim, since per-layer threat mapping exists [13]), a small measured study on a CPU-only stack (Section 8), and an evaluation plan. DAIRM defines no protocol, has not been validated on production or GPU systems, and has not been adopted by any standards development organisation. A fuller specification is maintained as a versioned technical report [14].

2 Background: Why Layered Reference Models Work

Layering as a structuring discipline predates networking. Dijkstra's THE system organised an operating system into levels, each relying only on services of the levels below [15], and Parnas argued that modules should hide design decisions likely to change [16]. The OSI model turned these ideas into a *reference* model for an industry [8, 10]. Three properties matter here.

Separation of concerns with stable boundaries. Each OSI layer provides a service to the layer above through a defined boundary, so implementations of one layer can change without changing their neighbours.

Named data units. OSI names what crosses each boundary. A named unit can be captured, decoded and measured, which is what makes protocol analysers effective: an engineer can inspect the unit at a boundary and assert which layer misbehaved.

Reference, not implementation. ISO describes ISO/IEC 7498-1 as a framework for coordinating standards that is not intended as an implementation specification [8]. The Internet's protocols do not map one-to-one onto OSI layers, yet the vocabulary stayed useful. The end-to-end argument [17] adds a caution that transfers directly to AI: some functions (e.g., correctness of the final answer) can only be implemented with knowledge held at the endpoints, so a layer model must not pretend that every concern is local to one layer.

Distributed tracing supplies the other ingredient. Dapper showed that propagating a trace context across service boundaries makes latency in large distributed systems attributable to individual components [18]; W3C Trace Context standardises trace-context propagation [19], and the OpenTelemetry generative AI semantic conventions, still at *Development* stability, define span attributes and metrics for generative AI operations [12]. DAIRM's diagnostic method is essentially a proposal for *which* boundaries such traces should mark in a generative AI system, and what state to record there.

3 Related Work and Independent Development

We group related work into six families and summarise the comparison in Table 1. The assessments are the author's reading of each work's public text or official description; ISO/IEC standards were assessed from their published scope statements, since full texts are not freely available. Corrections are welcome.

3.1 Independent development

DAIRM was developed independently by the author. A review of related work after its first public release (as "AOSI", v1.2.1, 9 November 2026) identified several earlier OSI-inspired layer models for AI: Reilly's "AOSI (AI Open Systems Interconnection) Reference Model" (GitHub, 28 December 2025), which uses the same name and a seven-layer framing with different layers [11]; Liang's seven-layer AI compute model from hardware to application (August 2025) [20]; Rodrigo-Ginés's OSI-inspired stack for agentic AI (September 2025) [21]; Cisco Research's proposed agent layers L8/L9 above OSI (November 2025) [22]; and the Cloud Security Alliance's MAESTRO, a seven-layer threat model for agentic AI (February 2025) [13]. To avoid confusion with Reilly's work, the model was renamed DAIRM in version 2.0.0. DAIRM does not claim the layered-stack idea or the OSI analogy as novel and does not claim priority over any of these works; its contributions are the three items listed in Section 1.

3.2 Families of related work

OSI-inspired and layered AI models. Reilly's AOSI has seven layers (infrastructure, model, data, orchestration, communication, interface, application) and states what each layer provides to the layer above, but defines no per-request boundary data state or diagnostic method [11]. Liang's

model spans the same hardware-to-application range with different cuts (physical, link, neural network, context, agent, orchestrator, application) and discusses KV caches and prefill/decode disaggregation within its layers [20]. Chander's seven-layer map places RAG inside orchestration and argues that training and serving need separate views [23]; DAIRM adopts both points through its context-services and build/deploy planes. Logan Smith's AI Layer Model groups nine layers into substrate, cognition and systems planes to place components and identify controls [24], and Darling's AILIS (v0.2.0, Dollhouse Research) proposes a finer 16+ layer vocabulary from facilities to application and asks adopters to name the hand-offs between layers, without itself defining them [25]. Localis uses "AI OSI Model" for a seven-layer technology taxonomy [26], and {a}OS describes seven agent-centric strata including a separate knowledge and memory stratum [27]. Rodrigo-Ginés layers interoperability between agents [21], the ACE framework layers an agent's cognition [28], and Fleming et al. propose agent communication and agent semantic layers (L8, L9) above the OSI application layer [22]; an earlier version of this work had reserved its own "L8/L9", which was removed to avoid that collision. None of these models defines a named, per-request data state at every boundary of the inference path or a latency-attribution method built on such states.

Software-architecture research on LLM and foundation-model systems. Lu et al. propose a taxonomy of foundation-model-based systems [29] and reference architectures for responsible FM-based systems [30] and agents [31]. Bucaioni et al. give a functional reference architecture for LLM-integrated systems and validate it against open-source systems [32], a level of validation DAIRM has yet to reach. Lin et al. propose a layered, model-native computing architecture with interface contracts between layers [33], close in spirit to named boundaries but at agent and operating-system level. Zhang et al. organise LLM-based software into model, inference and application layers and map desired capabilities onto them [34], and Tsaih et al. propose an AI tech-stack model for the management and technology challenges of AI-enabled application projects [35]. These works analyse design decisions, capabilities and components; DAIRM's unit of analysis is one request's execution path down to the device.

International standards and risk frameworks. ISO/IEC 22989 establishes AI concepts and terminology [36], and ISO/IEC 23053 describes the components and functions of ML-based AI systems [37]. The NIST AI RMF [38] and its Generative AI Profile [39] define organisational risk-management functions. These are broad or process-oriented; DAIRM can supply a technical location at which a "Measure" or "Manage" activity is scoped, and defers to ISO/IEC 22989 for general AI terms.

Industry stacks and MLOps. The a16z reference architecture for LLM applications [40] covers the application tier and treats the model as a hosted API. MLOps references [41, 42] are organised around the ML lifecycle; in DAIRM terms they sit upstream of the build/deploy plane.

Serving-systems research. Iteration-level scheduling [2], PagedAttention [3], chunked prefill [43] and prefill/decode disaggregation [44] show that the serving tier has its own bottlenecks. Surveys organise this space [45–47], roofline analysis explains why decode is often memory-bound [6], and MLPerf Inference provides system benchmarks [48]. These are taxonomies of optimisation techniques or measurements; DAIRM draws on them as evidence that the serving engine, runtime and hardware deserve separate layers.

AI security catalogues. The OWASP Top 10 for LLM Applications [49], MITRE ATLAS [50], NIST AI 100-2 [51] and Google's SAIF [52] describe what can go wrong and what to do. MAESTRO already maps threats by layer for agentic systems [13]; DAIRM's mapping (Section 6) is offered as content for the inference path, not as a novelty claim.

Table 1: Comparison of DAIRM with related work. ● = explicit, distinct element; ◐ = discussed, but not as a distinct layer, plane or boundary; ○ = not a focus. "HW" = hardware layer; "Serve" = inference-serving layer; "Ctx" = retrieval or context services; "States" = a named data representation at each boundary, capturable per request or per step; "Diag." = a method for attributing latency or failures to a tier; "Sec." = security threats or controls. Assessments are the author's reading of public descriptions (Section 3).

Work	Primary scope	HW	Serve	Ctx	States	Diag.	Sec.	Nature
Reilly, AOSI [11]	AI system functions, 7 layers	●	◐	◐	◐[b]	○	◐	Working draft
Liang [20]	AI compute architecture, 7 layers	●	◐	●	○	○	○	Preprint
Fleming et al. [22]	Agent protocol layers above OSI	○	○	○	○	○	○	Preprint
CSA MAESTRO [13]	Agentic AI threat model, 7 layers	◐	◐	●	○	○	●	Industry
Chander [23]	How LLMs work, 7 layers	○	●	◐	○	○	○	Blog
Smith, AI Layer Model [24]	AI component responsibilities, 9 layers	●	◐	●	○	○	◐	Practitioner
AILIS [25]	AI system vocabulary, 16+ layers	●	●	●	◐[d]	○	◐	White paper
Rodrigo-Ginés [21]	Multi-agent interoperability, 7 levels	●	◐	○	○	○	◐	Position paper
ACE [28]	Agent cognition, 6 layers	○	○	○	○	◐	○	Preprint
Bucaioni et al. [32]	LLM-integrated system functions	○	◐	◐	○	○	○	Preprint
Zhang et al. [34]	LLM software capabilities, 3 layers	○	◐	◐	○	○	○	Preprint
Lu et al. [30, 31]	FM-based system and agent patterns	○	◐	●	○	○	◐	Academic
ISO/IEC 22989, 23053 [36, 37]	AI concepts; ML system components	○	◐	○	○	○	◐	Standards
NIST AI RMF / 600-1 [38, 39]	Organisational AI risk process	○	○	○	○	○	◐	Guidance
a16z LLM stack [40]	LLM application tier	○	◐	●	○	○	◐	Industry
MLOps refs. [41, 42]	ML lifecycle (CI/CD/CT)	◐	◐	○	○	◐	○	Industry, academic
Serving surveys [45–47]	Serving optimisation techniques	◐	●	◐	○	○	○	Surveys
OWASP, ATLAS [49, 50]	Risk list; adversary techniques	○	◐	●	○	○	●	Community
DAIRM (this work)	**GenAI inference request path, L1–L6 + planes**	●	●	●	●	●[a]	●[c]	**Informative proposal**

[a]Proposed method; tested only in a small CPU-only study (Section 8). [b]Describes what each layer provides to the layer above (a service description), not a per-request data state. [c]Content, not a novelty claim. [d]Asks adopters to name hand-offs between layers but defines none.

Positioning. Table 1 shows that several models already cover the full vertical span from hardware to application, and several cover retrieval and security; DAIRM's span and its security table are therefore not distinctive. What no reviewed work provides is the "States" and "Diag." columns together: a named data state, joinable to individual requests, at every boundary of the inference path and a latency-attribution method that uses those states. That combination, plus the telemetry attributes that make it executable, is the whole of DAIRM's claim. Whether it is useful in practice is an empirical question that the small study of Section 8 only begins to address (Section 9.2).

4 The DAIRM Model

4.1 Scope

DAIRM models the request path of generative AI inference serving: from the client request to the hardware execution of each token, and back as a streamed response. It targets autoregressive transformer language models first, and multimodal models by extension. Training, fine-tuning, data pipelines, evaluation and the model registry are out of scope; they enter only through the build/deploy plane, as artifacts that the request path loads. Cluster orchestration, operating systems, containers and host configuration appear in a management plane but are not detailed. For hosted model APIs, layers below the gateway sit behind a *provider boundary* (Section 4.5).

4.2 Design principles

DAIRM's definitions follow seven principles: (P1) one primary responsibility per layer, with a product that spans layers described once per layer it occupies; (P2) a named, observable data state at every boundary, capturable per request or, where the serving engine batches requests, per scheduler step or device and joinable to the requests it served (metaphors and build-time artifacts do not qualify); (P3) adjacency of interfaces, with any other shortcut documented as an explicit cross-layer coupling; (P4) observability by construction; (P5) technology neutrality; (P6) *compute, data movement and energy concentrate in the lower layers, semantic and policy complexity concentrate in the upper layers, and latency can originate at any layer, so it is attributed per boundary rather than assumed*; and (P7) security controls placed at the layer or plane where a threat materialises.

4.3 Layers and planes

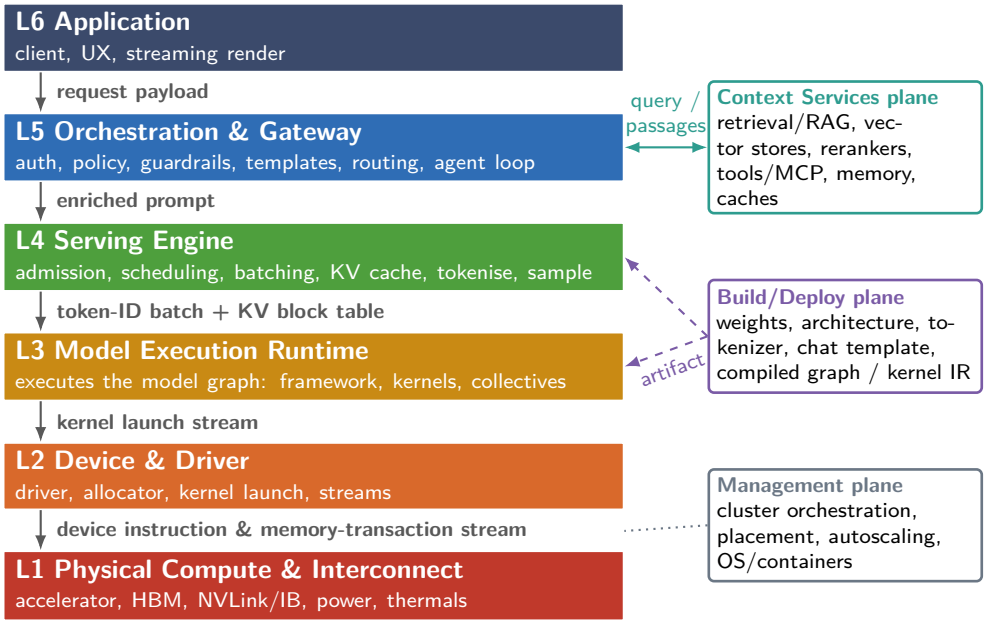


Figure 1: The DAIRM request path (L6–L1), the data state on each down-path boundary, and the three planes. Context services are called by L5 and return retrieved passages; the build/deploy plane supplies artifacts (dashed), which are not per-request data. The up-path returns logits → sampled token IDs → detokenised text deltas → streamed response chunks.

Figure 1 shows the model. We summarise each layer and plane by scope and characteristic failure modes; full component lists, interfaces and metrics are in the technical report [14].

L6 Application. User interaction, request serialisation and rendering of streamed output (chat UIs, IDE assistants, SDKs; SSE or WebSocket streaming). Failure modes: client timeouts shorter than server SLOs, proxies that buffer and defeat streaming, retry storms, rendering stalls misattributed to the model.

L5 Orchestration & Gateway. The policy enforcement point and the home of orchestration: authentication, quotas, model routing, prompt and chat templates, session state, input and output guardrails, the *agent loop* (planning, tool and function calling, MCP/A2A clients) and calls to context services. Failure modes: prompt bloat from unbounded history, guardrail latency dominating time-to-first-token (TTFT), output moderation that buffers the stream, template mismatch with the model's chat template, prompt injection passing policy checks, runaway agent loops.

L4 Serving Engine. Admission, queueing, iteration-level scheduling [2], KV-cache management [3], prefix caching, chunked prefill [43], disaggregation [44], tokenisation (executing the artifact's tokenizer) and sampling. Failure modes: KV-cache exhaustion leading to preemption and time-per-output-token (TPOT) spikes, head-of-line blocking from long prefills, over-batching beyond SLO, low prefix-cache hit rates caused by non-deterministic L5 prefixes.

L3 Model Execution Runtime. Executes the model graph supplied by the build/deploy plane for each scheduled batch, as framework operators, kernels (e.g., IO-aware attention [4]) and collectives. The model artifact *runs* here; it is not itself a layer. Failure modes: recompilation jitter, launch-bound execution at small batch, collective stalls from topology-unaware placement, numerical instability in low precision.

L2 Device & Driver. Driver and device runtime: memory allocation, kernel launch and streams, graph replay, partitioning and error reporting. Failure modes: driver/runtime incompatibility, out-of-memory from fragmentation, device resets. L2 is kept separate from L1 because driver faults (e.g., Xid-class errors) and physical limits need different responses.

L1 Physical Compute & Interconnect. Accelerators, memory hierarchy, intra- and inter-node fabrics, power and cooling. Memory bandwidth usually bounds decode speed [6]; that is a property of the workload, not a fault. Failure modes: thermal or power capping, link degradation, hardware errors.

Context Services plane. Retrieval and RAG [1], vector stores, embedding models, rerankers, tools and MCP servers, memory stores and caches, called by L5 and returning results. Retrieval is *not* a layer: not every request uses it, it is invoked and returns to orchestration, and the up-path never crosses it. Failure modes: retrieval lag, low recall, stale indexes, cross-tenant ACL leakage, poisoned documents.

Build/Deploy plane. Produces the artifacts that L4 and L3 load: weights and their numeric format, model architecture and configuration, tokenizer and chat template, and compiled graph or kernel IR (whether compiled ahead of time or at warm-up). Failure modes: configuration mismatch (e.g., positional-encoding scaling), quality regression from quantisation, tampered artifacts, stale compiled plans.

Ownership of cross-layer concerns. The tokenizer and chat template are artifacts executed at L4; L5 may pre-tokenise for budgeting, a documented coupling. Quantisation has one owner per

aspect: the artifact owns the format, L4 owns the deploy-time selection, and L3 kernels realise it. The KV cache is managed by L4, computed by L3 and held in L1 memory.

4.4 Boundary data states

Table 2: DAIRM boundary data states and their capture scope (P2). Request-scoped states belong to one request; step- and device-scoped states are shared by the requests batched together and are joined to them through `dairm.l4.step_id`. Activations and the KV cache are resident device-memory state, not boundary data; weights, tokenizer and compiled graph are build/deploy artifacts.

Boundary	**Data state (scope)**	**`dairm.data_state`**	**Observable as**
L6 → L5	Request payload (messages, tools, sampling parameters, stream flag); request	`request_payload`	Body, size, schema validity
Context → L5	Retrieved passages (text, scores, provenance); query embedding stays in the service; request	`retrieved_passages`	Passage count, tokens contributed, source IDs
L5 → L4	Enriched prompt (templated messages, tool schemas, sampling parameters, priority); request	`enriched_prompt`	Prompt tokens after enrichment
L4 → L3	Token-ID batch + KV-cache block table (token IDs, positions, sequence-to-block mappings); step	`token_batch`	Batch size, tokens per step, KV blocks used
L3 → L2	Kernel launch stream (operator and kernel invocations with tensor arguments); step	`kernel_stream`	Kernel traces, launch counts
L2 → L1	Device instruction and memory-transaction stream; device, per interval	`device_stream`	SM and tensor-pipe activity, DRAM and link bytes
Up-path	Logits → sampled token IDs → text deltas → response chunks; request	`token_stream`	Inter-token latency, stream events

Table 2 lists the states. Three design choices distinguish them from a naive "one artefact per layer" chain. First, every state is *capturable and measurable at a stated scope*: per request above the serving engine, and per scheduler step or device below it, where one batch or one kernel serves several requests and a step identifier joins the two; physical metaphors are excluded, and the lowest boundary is defined by what hardware counters can observe. Second, *retrieval returns text*: what crosses back to orchestration is passages with scores and provenance, not vectors, because embeddings are internal to the context service and the serving engine builds its batch from the token IDs of the enriched text. Third, *resident state and artifacts are kept off the path*: activations and the KV cache are produced and consumed in place by L3 kernels running on L1, and model weights and compiled graphs are loaded from the build/deploy plane, so no state is shown flowing "down" before it has been computed.

Two properties make the states useful rather than decorative. Each has a measurable size that propagates downward: enriched-prompt tokens at L5/L4 determine prefill work below; passages and tokens contributed at Context/L5 determine part of that count; batch composition at L4/L3 determines the arithmetic intensity seen at L1. And each is a *checkpoint for blame*, used as a heuristic rather than a rule: if the state entering a layer is unchanged and the layer's span grew, that

layer is the first suspect; if the incoming state changed, the layer above is suspected. Contention from co-located work, or a degraded lower layer, can lengthen a span while its input stays the same, so the heuristic is always followed by lower-layer checks and an isolation test (Section 5.4).

4.5 Provider boundary

For hosted model APIs, L4–L1 (and often parts of L5) are opaque to the consumer. What stays observable is client-side TTFT and inter-token latency, the consumer's own orchestration and context-service spans, token usage returned by the API, and any provider-reported latency or cache fields. The decomposition in Section 5 still separates the consumer's own time from time behind the boundary, and isolation tests can show whether a regression is inside or outside it, though not which provider-side layer caused it.

5 Boundary-Attributed Diagnostics

5.1 Principle

A protocol analyser makes network faults tractable by decoding a captured unit layer by layer. DAIRM proposes the analogous discipline for generative AI: *each request should produce a trace in which every span is attributed to exactly one layer or plane, and each boundary hand-off is timestamped together with its data state.* Concretely, a DAIRM-aligned deployment propagates one trace context from L6 through L4 and into context-service calls [18, 19]; tags each span with the attributes of Section 5.5 alongside OpenTelemetry generative AI attributes [12]; exports serving-engine metrics [53] and device metrics [54] on a common time base with node, device and replica labels; and captures runtime profiles on demand.

5.2 Latency decomposition

For a streamed response with N_{out} output tokens,

$$E2E \approx \text{TTFT} + (N_{\text{out}} - 1) \cdot \text{TPOT}, \tag{1}$$

$$\text{TTFT} \approx t_{L6} + t_{L5}^{\text{gw}} + t_{L5}^{\text{guard,in}} + t_{\text{ctx}} + t_{L4}^{\text{queue}} + t_{L4}^{\text{tok}} + t_{\text{prefill}}^{L3\text{–}L1} + t_{\text{up}}, \tag{2}$$

$$\text{TPOT} \approx t_{L4}^{\text{sched}} + t_{\text{decode}}^{L3\text{–}L1} + t_{L5}^{\text{guard,out}}. \tag{3}$$

Here TPOT is a request's mean decode time per output token, $(\text{E2E} - \text{TTFT})/(N_{\text{out}} - 1)$; inter-token latency (ITL) is the distribution of individual gaps, which exposes stalls that a mean hides, and engines such as vLLM export both [53]. TTFT inflated with TPOT normal means the cause is upper-layer time (gateway, guardrails, context services), *or* L4 queueing, *or* prefill compute for longer prompts; the three are separated by the L5 and context spans, the L4 queue span versus the prefill span, and the prompt-token count at L5/L4. TPOT inflated with TTFT normal points at L4 decode (KV pressure, batch growth), L3 kernels (attention cost grows with context; collectives), L1 memory bandwidth or clock events, or buffering output guardrails in L5. Both inflated suggests overload, device degradation or retry storms.

5.3 Discriminating three commonly confused causes

Table 3 lists signals that separate context-service delay, L4 KV-cache pressure and memory-bandwidth-bound decode, each of which surfaces as "the model is slow". The signatures follow from

the mechanisms in the serving and roofline literature [3, 6]; they are hypotheses to be calibrated per deployment, not measured thresholds.

Table 3: Discriminating signals for three causes of perceived slowness. Derived from mechanism, not measurement; calibrate per deployment.

	Context-service delay	**L4 KV-cache pressure**	**Memory-bandwidth-bound decode**
Latency signature	TTFT up; TPOT unchanged	TPOT up and spiky; TTFT up if admission blocks	TPOT up smoothly with batch and context
Trace location	Context span grows before any L4 span	L4 queue span grows; decode shows preemption gaps	L4 decode span grows uniformly per step
Engine / device signals	Engine and device normal	KV usage near capacity; preemption counters rise	DRAM utilisation high; tensor-pipe utilisation comparatively low
Isolation test	Replay with context services bypassed and an equal-length fixed context	Replay with lower concurrency or a larger KV budget	Sweep context length at fixed batch, and batch at fixed context; check that TPOT tracks bytes read per step
Conclusion if test reproduces	Context services implicated	L4 implicated	*Consistent with* bandwidth-bound decode (attention cost at L3 also grows with context)
Remediate at	Index residency, co-location, smaller k	Prefix caching, KV quantisation, admission limits, larger KV budget	Weight/KV quantisation, speculative decoding, scale-out

The third column is deliberately hedged. High memory-bandwidth use during decode is usually a property of the workload's arithmetic intensity, not a hardware fault, and attention-kernel cost and KV reads also grow with context, so a context sweep can show behaviour consistent with bandwidth-bound decode but cannot by itself prove an L1 cause.

5.4 Triage procedure

Diagnosis proceeds top-down for attribution and bottom-up for confirmation: (1) quantify the symptom (TTFT, TPOT, E2E, error rate, goodput or quality; percentile and window); (2) decompose it into TTFT and TPOT movement; (3) walk the trace top-down (L6→L5/context→L4) against a baseline—the first layer or plane whose span distribution shifted is the *candidate*; (4) check the boundary state entering the candidate (prompt tokens at L5/L4, passages at Context/L5, batch size at L4/L3), since a changed input points, heuristically, to the layer above; (5) correlate with lower-layer telemetry (engine metrics, runtime profiles, device counters); (6) run the candidate's isolation test, changing one variable; (7) confirm bottom-up that lower layers are healthy; (8) remediate at the owning layer or plane and re-measure; and (9) record the incident with its root-cause and manifestation layers. Step 9 is what would make DAIRM testable at scale (Section 9.2).

5.5 Telemetry attributes

Table 4 summarises the proposed attributes, designed to complement the OpenTelemetry generative AI conventions [12], which are at *Development* stability and may change. Where those conventions already identify an operation, they take precedence: `gen_ai.operation.name` (e.g., `retrieval`, `execute_tool`, `invoke_agent`) names what a span does, and `dairm.plane` and `dairm.layer` only

add where it sits. Likewise, TTFT and TPOT map to the `gen_ai.server.time_to_first_token` and `gen_ai.server.time_per_output_token` metrics, and durations use seconds, following OpenTelemetry practice. Engine and device names vary by version; for example, current vLLM documentation lists `vllm:kv_cache_usage_perc` and `vllm:num_preemptions` [53], and the current DCGM field list uses `DCGM_FI_PROF_DRAM_UTIL_RATIO` [54]; implementers should pin versions.

Table 4: Proposed `dairm.*` telemetry attributes (selection; full list in the technical report [14]).

Attribute	Where	Meaning
`dairm.layer` (int 1–6)	Request path	Layer to which the span is attributed
`dairm.plane`	All spans	`request_path`, `context_services`, `build_deploy` or `management`
`dairm.data_state`	Boundary spans	One of the values in Table 2
`dairm.l5.prompt_tokens_enriched`	L5	Prompt tokens after templating, history and passages
`dairm.ctx.service`, `dairm.ctx.tokens_contributed`	Context services	Kind of service; tokens its results add to the prompt
`dairm.l4.queue_time` (s), `dairm.l4.preempted`	L4	Queueing before first scheduling; whether the sequence was preempted
`dairm.l4.step_id`	L4, L3	Scheduler step that executed a batch; joins step- and device-scoped data to requests
`dairm.device_id`, `dairm.artifact.id`	L2/L1; build/deploy	Join keys to device telemetry and to the model artifact

6 Security Mapping

Under principle P7, each threat is placed at the layer or plane where it materialises. Table 5 maps the OWASP Top 10 for LLM Applications (2025) [49] onto DAIRM. Per-layer threat mapping is not new [13]; the mapping is the author's, offered as content and intended to be contested. Two observations follow. First, several risks are multi-location by nature: prompt injection is *enforced against* at L5 but *arrives* via L6 (direct) or through context services (indirect, via retrieved content [55]); misinformation is the joint outcome of grounding, the model artifact and presentation. Second, a location index exposes control gaps that a flat list hides: an architecture whose only control for LLM04 and LLM08 is an L5 guardrail has no control where poisoning materialises (the corpus or the weights).

Beyond OWASP, the location index surfaces threats that application-level lists tend not to emphasise: embedding inversion, which can recover text from stored vectors in context services [56]; cross-request leakage through shared prefix or KV caches via timing side channels at L4 [57]; unsafe deserialisation of model files at L3; and hardware side channels or memory residue between tenants at L2/L1.

7 Illustrative Worked Example

Note. This section is a constructed, hypothetical walk-through intended only to show how the vocabulary and procedure are applied. It does not describe a real incident and reports no measurements.

Setting. An internal question-answering assistant: a web client (L6) calls an AI gateway that applies a system prompt, conversation history and guardrails (L5); the gateway retrieves policy documents from a vector index (context services); a self-hosted serving engine with continuous

Table 5: OWASP Top 10 for LLM Applications (2025) mapped to DAIRM locations, with representative controls. Author's mapping; illustrative.

OWASP risk (2025)	**Primary location(s)**	**Representative controls**
LLM01 Prompt Injection	L5 (arrives via L6 or context)	Separation of system, user and retrieved content; input classifiers; least-privilege tools; content scanning in context services
LLM02 Sensitive Information Disclosure	L5, context, build	Output redaction (L5); ACL-filtered retrieval (context); training-data extraction testing (build)
LLM03 Supply Chain	Build, L3	Signed, hash-verified artifacts and provenance; pinned dependencies and SBOMs
LLM04 Data and Model Poisoning	Build (weights), context (corpus)	Model provenance and red-teaming; ingestion integrity checks
LLM05 Improper Output Handling	L5, L6	Schema validation (L5); rendering that never executes untrusted content (L6)
LLM06 Excessive Agency	L5 (agent loop), context (tools)	Tool permission scoping, human approval gates, action budgets
LLM07 System Prompt Leakage	L5	Secrets kept out of prompts; leakage detection on output
LLM08 Vector and Embedding Weaknesses	Context	Vector-store access control; tenant isolation; encryption at rest
LLM09 Misinformation	Context, build, L6	Grounding evaluation and freshness; quality evaluation; citation display
LLM10 Unbounded Consumption	L4, L5	Token and time budgets (L4); rate limits and quotas (L5)

batching and a paged KV cache (L4) runs a quantised model artifact (build/deploy) through a standard runtime (L3) on a multi-GPU node (L2/L1).

Symptom A: "the assistant got slow this week". Step 1 restates the complaint as a metric: suppose p95 TTFT has risen relative to the previous week while TPOT has not. Step 2 therefore points at upper-layer time, L4 queueing or prefill. Walking the trace (Step 3), suppose the context-service span is unchanged but the L4 queue and prefill spans have grown. Step 4 checks the boundary state at L5/L4 and finds that enriched-prompt token counts have grown, because a recent L5 change now replays full conversation history instead of a summary. The input to L4 changed, so the layer *above* is implicated: the root cause is at L5, although the time is spent at L4–L1. The isolation test (Step 6) replays the same requests with history truncated; if TTFT returns to baseline, the attribution is confirmed. The remediation is at L5 (history windowing or summarisation), not at L4 (more replicas), which would only have masked the cause. The incident is recorded as *L5, prompt growth, manifesting at L4*.

Symptom B: "the assistant cites the wrong policy". Latency is normal, so the decomposition is uninformative and the operators move to quality signals. The boundary states order the questions: were the *retrieved passages* returned to L5 the right documents (a context-services question)? Did they reach the *enriched prompt* intact and in the intended position (an L5 template question)? Given correct context, did the model use it (a question about the build/deploy artifact, testable by replaying the identical enriched prompt against a reference-precision artifact)? Each question isolates one location by holding the incoming state fixed. If the index had not been refreshed after a policy update, the fault is a context-service freshness failure, which also corresponds to OWASP

LLM09 and is fixed there rather than by a stricter L5 output filter.

What the example shows and does not show. It shows that the vocabulary distinguishes *where time is spent* from *where the cause lies*, and that boundary states give a principled order of questions. It does not show that practitioners using DAIRM diagnose faster or more accurately; that is the hypothesis the evaluation plan tests. A measured version of Symptom A, on a small CPU stack, is in Section 8.

8 A Small Measured Study on a CPU-Only Stack

To check that the procedure of Section 5.4 can be executed on real telemetry, and that it attributes faults to where they were injected, we ran a small fault-injection study. Code, raw per-request records, telemetry samples, OpenTelemetry spans and the analysis script are in the repository's `eval/` folder; every number below is computed from those files.

Setup. One cloud VM with 8 vCPUs ("Intel Xeon Processor"), 15.6 GB RAM and no GPU. L4 is llama.cpp `llama-server` (release b12239) serving Qwen2.5-0.5B-Instruct (GGUF, Q4_K_M) with continuous batching over 4 slots and 4 threads pinned to CPUs 0–3; L3–L1 is its CPU backend on those cores. L5 is a Python gateway (input guardrail, retrieval call, chat template, conversation-history policy, token counting) and CTX is an HTTP TF-IDF retrieval service over a synthetic policy corpus; both, and the closed-loop L6 client, run on CPUs 4–7. Each request emits OpenTelemetry spans tagged with `dairm.layer`, `dairm.plane`, `dairm.data_state`, `dairm.l5.prompt_tokens_enriched` and `dairm.l4.queue_time`; a sampler records the engine's `requests_deferred` gauge and the Linux PSI CPU stall fraction every 0.25 s.

Procedure. Each of 3 repetitions measures a baseline and then, in seeded random order, five conditions of 24 requests (concurrency 3, 96 output tokens): no fault (control); a 1000 ms delay in the CTX service; an L5 change that replays ten earlier turns instead of a summary; an L4 change from 4 slots to 1 at the same load and per-slot KV budget; and four busy-loop processes pinned to the engine's cores ("L2/L1" here). An automated triage encodes Steps 2–5 against the same repetition's baseline: a metric moved if its median changed by more than 25% and by an absolute floor; the first span that moved, top-down, is the candidate; if it is at L4 or below, a grown enriched-prompt count attributes the root cause to L5, a risen deferred gauge to L4, and a risen CPU stall to L2/L1. It then runs the isolation test its attribution selects. The triage never reads the condition label. Thresholds were fixed before the reported run; two one-repetition pilots led to longer outputs and a larger CTX delay after the slot fault (pilot 1) and a 400 ms CTX delay (pilot 2) were missed, which we report as a condition of the result.

Results. Table 6 summarises the run (2026-11-10, about 61 minutes). The triage attributed the root cause correctly in all 15 condition-repetitions (12 of 12 faults, 3 of 3 controls), and the selected isolation test confirmed each fault attribution. The informative cases are the six runs in which the first span to move was at L4 although the fault was not: under the L5 history change, prompt tokens at the L5/L4 boundary rose from a median of 288 to 1118, and under core contention the boundary state was unchanged while the CPU stall rose and the engine's deferred gauge did not. A rule of "blame the first span that grew" would have blamed L4 in those six runs; the boundary-state check is what moved the attribution, which is the behaviour Section 4.4 predicts. Tails were heavy under contention (the slowest request's TTFT was about 111 s), so the triage uses medians.

Table 6: Measured results: median over 3 repetitions of per-repetition medians (24 requests each), and attribution outcomes. TTFT is client-observed; spans are from the DAIRM-tagged traces.

Condition	**TTFT**	**TPOT**	**CTX**	**L4 queue**	**Prefill**	**Prompt**	**Attributed root (3 reps); isolation test confirms**
	ms	ms	ms	ms	ms	tokens	
Baseline	1812	23.5	3	14	1803	288	—
No fault (control)	1806	23.9	2	12	1751	288	none, none, none
CTX delay 1000 ms	2819	23.8	1002	17	1804	288	CTX ×3; 3/3
L5 full history	7038	45.7	2	50	4247	1118	L5 ×3 (manifest L4); 3/3
L4 slots 4→1	4132	12.6	2	3551	579	288	L4 ×3; 3/3
Engine-core contention	4459	159.6	3	26	4405	288	L2/L1 ×3 (manifest L4); 3/3

What this does and does not show. It shows that the boundary states and `dairm.*` attributes can be produced on a real serving stack and that a mechanical encoding of the procedure separates five single, sufficiently large faults on that stack. It does not test the KV-pressure or memory-bandwidth discriminators of Table 3: the engine exposes no KV-usage or preemption metric, and on a CPU, L3, L2 and L1 are one block whose "device" signal (PSI stall) is close to the injected mechanism. It uses one model, one machine and a synthetic workload; the same author designed the faults, the triage and the analysis, with ground truth known when the pilots were adjusted; and it measures the procedure, not whether people diagnose faster with DAIRM.

9 Discussion, Limitations and Evaluation Plan

9.1 Limitations and threats to validity

- **Limited empirical validation.** Apart from the small CPU-only study of Section 8, the boundaries, discriminating signals and security mapping are argued from mechanism and the literature, not measured; the KV-pressure and memory-bandwidth discriminators are untested. Thresholds in the technical report are explicitly illustrative.
- **Crowded design space.** Many OSI-inspired AI models exist (Section 3). DAIRM's claim rests only on the boundary states, the diagnostic method and the telemetry attributes; if a prior work providing these is found, the claim narrows accordingly.
- **No standards adoption.** DAIRM is a single-author proposal. It has not been reviewed or adopted by ISO/IEC, ITU-T, the IETF, NIST or any other body.
- **Construct validity of boundaries.** Real systems violate strict layering: serving engines ship their own kernels (L4/L3), gateways cache responses, and embedding models used by context services are themselves served by an L4 engine. DAIRM handles these through P1 (describe a component once per layer it occupies), P3 (document couplings) and nested traces, but whether this stays workable for real inventories is untested.
- **Planes versus layers.** Treating retrieval as a plane fixes the ordering problem of a "retrieval layer" but makes context services less visible in a simple stack diagram. Long-context designs without retrieval and in-model retrieval simply do not use the plane.
- **Scope.** Training, fine-tuning and data pipelines are excluded, and multi-step agent workflows are modelled only as repeated passes through L5. Many production issues arise exactly there.

- **Author bias and AI assistance.** The comparison in Table 1 and the mapping in Table 5 were produced by the model's author with AI-assisted drafting; independent replication of both is needed.

9.2 Evaluation plan

We propose four studies, ordered by cost.

1. **Incident classification study.** Collect public post-incident reports and issue-tracker reports from open-source serving engines, gateways and RAG frameworks. Two or more independent raters classify each by DAIRM location of *root cause* and of *manifestation*, using a written codebook. Outcomes: inter-rater agreement (e.g., Cohen's or Fleiss' κ), the fraction of reports that cannot be assigned (a coverage test), and how often root cause and manifestation differ.
2. **Practitioner card-sorting.** Ask practitioners across application, platform, ML and infrastructure roles to sort component and symptom cards into layers and planes and rate each boundary's clarity. Outcomes: agreement with DAIRM's assignments and boundaries with systematic disagreement.
3. **Instrumented GPU case study.** Section 8 is a CPU-only first step. Repeat it on GPUs with a production engine (e.g., vLLM) and DCGM counters, inject KV-cache pressure through concurrency or long contexts and bandwidth-bound decode as well as combined and smaller faults, and measure whether the discriminators of Table 3 attribute each fault correctly, reporting measured thresholds; ideally with faults chosen by someone other than the author.
4. **Controlled diagnosis experiment.** Compare time-to-correct-attribution and accuracy for participants with and without DAIRM-tagged traces on the same injected faults. This is the only study that would test the central practical claim.

Results from any of these, including negative results, would warrant revision of the model.

9.3 Future work

An earlier version of this work reserved layers "L8" (agentic workflows) and "L9" (distributed inference) above the stack; these were removed because Fleming et al. had already proposed L8/L9 agent layers above OSI [22]. Instead, future work will (a) model multi-step agent executions as repeated passes through the request path with nested L5 spans; (b) model distributed and disaggregated inference within L4–L1, adding an inter-node KV-transfer boundary state; (c) add a separate training/build view that meets the request path at the model artifact, as Chander argues is necessary [23]; and (d) define cross-layer efficiency metrics such as energy per output token.

10 Conclusion

Generative AI inference spans tiers that are engineered, observed and secured separately, and practitioners lack a shared way to say where a problem lives. Many layered and OSI-inspired models for AI already exist, and DAIRM claims none of the general idea. Its contribution is narrower: a named, observable data state at every boundary of the inference request path; a boundary-attributed diagnostic method that uses those states to separate context-service delay, serving-engine queueing or KV-cache pressure, and memory-bandwidth-bound decode; and telemetry attributes that make the method executable. Beyond a small CPU-only study in which an automated triage located 12

of 12 injected faults, the model is unvalidated, and its value depends on whether practitioners can apply it consistently and whether it shortens diagnosis. We have outlined studies to test both and invite counter-examples, alternative boundaries and missed prior art.

Acknowledgements and AI-Assistance Disclosure

Drafting, editing, formatting and figure preparation were assisted by AI tools. The conceptual model, design decisions and final content are the author's, and the author takes full responsibility for them. All figures are drawn in TikZ from source; no figure is an AI-generated image.

Availability

The technical report DAIRM-TR-0001 (version 2.0.0), including per-layer interfaces, metrics, failure modes, a troubleshooting matrix and the full telemetry attribute list, is maintained at `https://github.com/CodingThrill0012/dairm` under CC-BY-4.0 and archived under the concept DOI `https://doi.org/10.5281/zenodo.24073121` (version 2.0.0: `https://doi.org/10.5281/zenodo.24123645`) [14]. It supersedes AOSI v1.2.1 (`https://doi.org/10.5281/zenodo.24073122`).

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*, 2020. URL `https://arxiv.org/abs/2005.11401`.

[2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022. URL `https://www.usenix.org/conference/osdi22/presentation/yu`.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL `https://arxiv.org/abs/2205.14135`.

[5] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[6] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024. URL `https://arxiv.org/abs/2402.16363`.

[7] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503–2511, 2015.

[8] ISO/IEC. ISO/IEC 7498-1:1994, Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model. International Organization for Standardization, 1994. URL `https://www.iso.org/standard/20269.html`.

[9] ITU-T. Recommendation X.200: Information technology – Open Systems Interconnection – Basic Reference Model: The basic model. International Telecommunication Union, 1994. URL `https://www.itu.int/rec/T-REC-X.200`.

[10] Hubert Zimmermann. OSI reference model – the ISO model of architecture for open systems interconnection. *IEEE Transactions on Communications*, 28(4):425–432, 1980. doi: 10.1109/TCOM.1980.1094702.

[11] Shawn Reilly. AOSI Reference Model for Artificial Intelligence (AOSI = AI Open Systems Interconnection), version 1.0. GitHub repository kahalewai/aosi (created 28 December 2025), 2025. URL `https://github.com/kahalewai/aosi`.

[12] OpenTelemetry. Semantic Conventions for Generative AI. GitHub repository open-telemetry/semantic-conventions-genai. URL https://github.com/open-telemetry/semantic-conventions-genai. Accessed 9 November 2026.

[13] Ken Huang. Agentic AI Threat Modeling Framework: MAESTRO. Cloud Security Alliance blog, 6 February, 2025. URL https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro.

[14] Divyendu Bhatt. DAIRM: Divyendu's data-state-aware AI reference model for generative AI inference systems. Technical Report / Working Draft DAIRM-TR-0001, version 2.0.0, Independent, November 2026. Concept DOI covering all versions; version 2.0.0: doi:10.5281/zenodo.24123645; supersedes AOSI v1.2.1, doi:10.5281/zenodo.24073122.

[15] Edsger W. Dijkstra. The structure of the "THE"-multiprogramming system. *Communications of the ACM*, 11 (5):341–346, 1968. doi: 10.1145/363095.363143.

[16] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[17] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984. doi: 10.1145/357401.357402.

[18] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Technical Report dapper-2010-1, Google, 2010. URL https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/.

[19] W3C. Trace Context. W3C Recommendation. URL https://www.w3.org/TR/trace-context/.

[20] Bor-Sung Liang. AI compute architecture and evolution trends. *arXiv preprint arXiv:2508.21394*, 2025. URL https://arxiv.org/abs/2508.21394.

[21] Francisco-Javier Rodrigo-Ginés. Toward an OSI-Inspired Stack for Agentic AI: From Fragmented Frameworks to Open Standards. Zenodo, 2025.

[22] Charles Fleming, Luca Muscariello, Vijoy Pandey, and Ramana Kompella. A layered protocol architecture for the internet of agents. *arXiv preprint arXiv:2511.19699*, 2025. URL https://arxiv.org/abs/2511.19699.

[23] C. P. Chander. How LLMs Actually Work: A Seven-Layer Map. Blog post, 19 September 2026, 2026. URL https://cpchander.com/blog/how-llms-work-seven-layer-map.

[24] Logan Smith. The AI Layer Model: a shared map for AI systems. Blog post, 20 October 2026; repository github.com/Logans415/ai-layer-model, 2026. URL https://logansmith.us/post-ai-layer-model.html.

[25] Mick Darling. AILIS: A proposed layer model for AI systems. Version 0.2.0, Dollhouse Research; GitHub repository DollhouseMCP/AILIS, 2025. URL https://ailis.dollhouseresearch.com/. Accessed 9 November 2026.

[26] Localis. AI OSI Model. URL https://localis.services/ai-osi/. Accessed 9 November 2026.

[27] {a}OS Explorer. The OSI Model for AI. URL https://aos7.tech/. Accessed 9 November 2026.

[28] David Shapiro, Wangfan Li, Manuel Delaflor, and Carlos Toxtli. Conceptual framework for autonomous cognitive entities. *arXiv preprint arXiv:2310.06775*, 2023. URL https://arxiv.org/abs/2310.06775.

[29] Qinghua Lu, Liming Zhu, Xiwei Xu, Yue Liu, Zhenchang Xing, and Jon Whittle. A taxonomy of foundation model based systems through the lens of software architecture. *arXiv preprint arXiv:2305.05352*, 2023. URL https://arxiv.org/abs/2305.05352.

[30] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, and Jon Whittle. A reference architecture for designing foundation model based systems. *arXiv preprint arXiv:2304.11090*, 2023. URL https://arxiv.org/abs/2304.11090.

[31] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, Stefan Harrer, and Jon Whittle. Towards responsible generative AI: A reference architecture for designing foundation model based agents. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, pages 119–126, 2024. doi: 10.1109/ICSA-C63560.2024.00028.

[32] Alessio Bucaioni, Martin Weyssow, Junda He, Yunbo Lyu, and David Lo. A functional software reference architecture for LLM-integrated systems. *arXiv preprint arXiv:2501.12904*, 2025. URL https://arxiv.org/abs/2501.12904.

[33] Hai Lin, Hoilam Pao, Shaoxiong Zhan, and Hai-Tao Zheng. Model-native computing architecture: Envisioning future system architecture through the lens of computer architecture. *arXiv preprint arXiv:2606.00288*, 2026. URL https://arxiv.org/abs/2606.00288.

[34] Dawen Zhang, Xiwei Xu, Chen Wang, Zhenchang Xing, and Robert Mao. A layered architecture for developing and enhancing capabilities in large language model-based software systems. arXiv preprint arXiv:2411.12357, 2024. URL https://arxiv.org/abs/2411.12357.

[35] Rua-Huan Tsaih, Hsin-Lu Chang, Chih-Chun Hsu, and David C. Yen. The AI tech-stack model. *Communications of the ACM*, 66(3):69–77, 2023. doi: 10.1145/3568026.

[36] ISO/IEC JTC 1/SC 42. ISO/IEC 22989:2022, Information technology – Artificial intelligence – Artificial intelligence concepts and terminology. International Organization for Standardization, 2022. URL `https://www.iso.org/standard/74296.html`.

[37] ISO/IEC JTC 1/SC 42. ISO/IEC 23053:2022, Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). International Organization for Standardization, 2022. URL `https://www.iso.org/standard/74438.html`.

[38] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, 2023. URL `https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf`.

[39] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. Technical Report NIST AI 600-1, NIST, 2024.

[40] Matt Bornstein and Rajko Radovanovic. Emerging Architectures for LLM Applications. Andreessen Horowitz, 2023. URL `https://a16z.com/emerging-architectures-for-llm-applications/`.

[41] Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning. Cloud Architecture Center. URL `https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning`. Accessed 9 November 2026.

[42] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. doi: 10.1109/ACCESS.2023.3262138.

[43] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023. URL `https://arxiv.org/abs/2308.16369`.

[44] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024.

[45] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023. URL `https://arxiv.org/abs/2312.15234`.

[46] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. LLM inference serving: Survey of recent advances and opportunities. *arXiv preprint arXiv:2407.12391*, 2024. URL `https://arxiv.org/abs/2407.12391`.

[47] James Pan and Guoliang Li. A survey of LLM inference systems. *arXiv preprint arXiv:2506.21901*, 2025. URL `https://arxiv.org/abs/2506.21901`.

[48] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, et al. MLPerf inference benchmark. *arXiv preprint arXiv:1911.02549*, 2019. URL `https://arxiv.org/abs/1911.02549`.

[49] OWASP GenAI Security Project. OWASP Top 10 for LLM Applications 2025, 2025. URL `https://genai.owasp.org/llm-top-10/`.

[50] The MITRE Corporation. ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems. URL `https://atlas.mitre.org/`. Accessed 9 November 2026.

[51] Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. Technical Report NIST AI 100-2 E2025, NIST, 2025.

[52] Google. Secure AI Framework (SAIF). URL `https://saif.google/`. Accessed 9 November 2026.

[53] vLLM Project. Metrics. vLLM documentation. URL `https://docs.vllm.ai/en/latest/usage/metrics.html`. Accessed 9 November 2026.

[54] NVIDIA. DCGM API: Field IDs. Data Center GPU Manager documentation. URL `https://docs.nvidia.com/datacenter/dcgm/latest/dcgm-api/dcgm-api-field-ids.html`. Accessed 9 November 2026.

[55] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL `https://arxiv.org/abs/2302.12173`.

[56] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text embeddings reveal (almost) as much as text. *arXiv preprint arXiv:2310.06816*, 2023. URL `https://arxiv.org/abs/2310.06816`.

[57] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. The early bird catches the leak: Unveiling timing side channels in LLM serving systems. *arXiv preprint arXiv:2409.20002*, 2024. URL `https://arxiv.org/abs/2409.20002`.